

Laboratoire n° 8

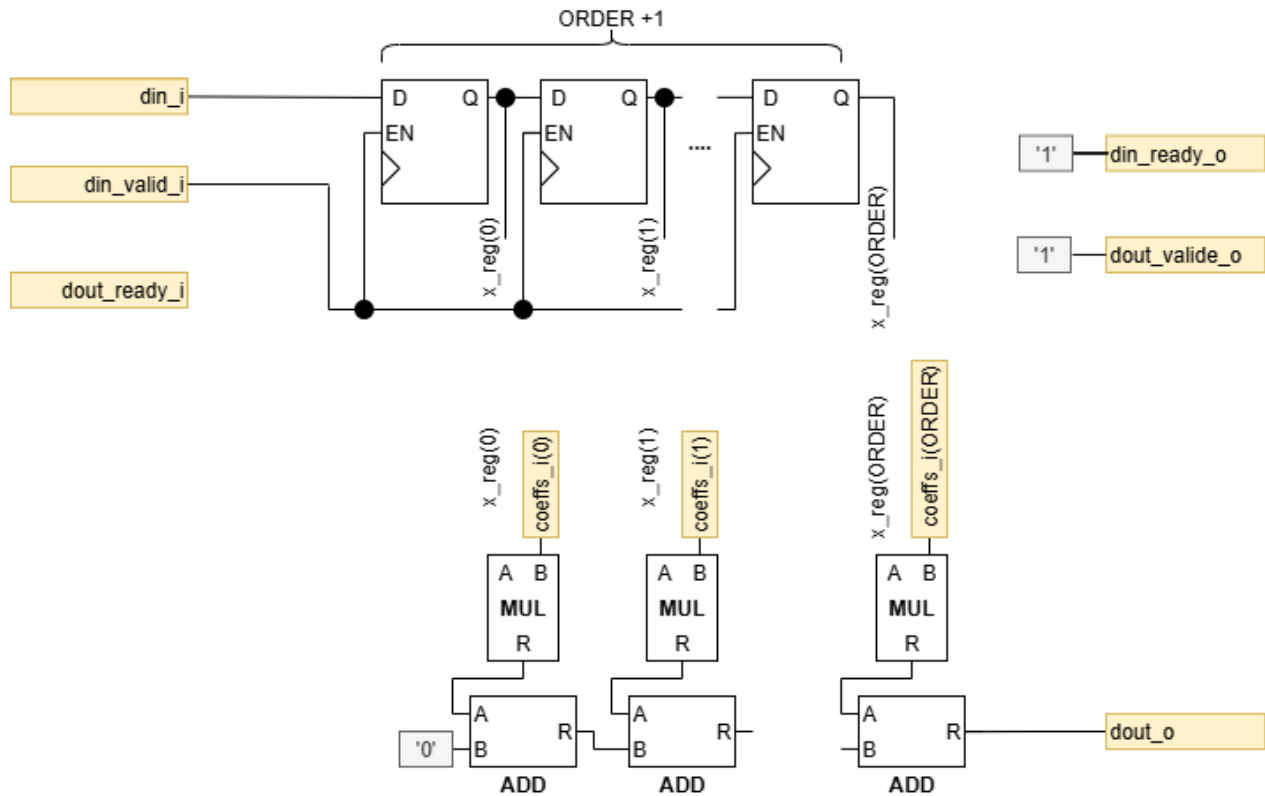
Par : Rodrigo Lopes dos Santos

Par : Urs Behrmann

Filtre FIR : Combinatoire

Le filtre FIR est implémenté sous forme combinatoire, avec pour objectif de maximiser le débit. Les multiplications et additions nécessaires au calcul sont réalisées en parallèle.

Schéma



Chemin de données (datapath)

Les données d'entrée din_i sont stockées dans un registre à décalage (x_reg) de taille $ORDER+1$, permettant de conserver les derniers échantillons reçus.

Chaque échantillon $x_reg(k)$ est multiplié en parallèle avec son coefficient associé $coeffs(k)$. Les produits obtenus sont ensuite additionnés par une chaîne d'additionneurs afin de produire directement le résultat FIR.

Le résultat final est décalé selon $COMMAPOS$, puis envoyé sur $dout_o$.

Unité de contrôle

Cette architecture ne nécessite pas de FSM, car le calcul est effectué directement par la logique combinatoire.

Le registre à décalage est mis à jour à chaque front montant de l'horloge. Le calcul FIR est ensuite automatiquement mis à jour en fonction du contenu de x_reg .

Interface

Dans cette version, le filtre accepte une donnée à chaque cycle :

- `din_ready_o` = 1 en permanence → nouvelle donnée toujours acceptée
- `dout_valid_o` = 1 en permanence → sortie toujours considérée disponible

Le signal `dout_ready_i` n'est pas utilisé dans cette architecture, car la sortie est recalculée en continu.

Latence & débit

La latence est très faible : elle correspond principalement au cycle nécessaire pour charger l'entrée dans le registre à décalage, puis au temps de propagation de la logique combinatoire.

Le débit est maximal :

$$\text{Débit} = 1 \text{ sortie / cycle}$$

Cette architecture privilégie donc la rapidité et le débit, au prix d'une utilisation plus importante des ressources matérielles.

Fréquence d'horloge

Dans l'architecture combinatoire, l'ensemble du calcul FIR (multiplications et additions) est réalisé en un seul cycle d'horloge. Le chemin critique est donc très long, car il traverse toutes les opérations arithmétiques en cascade.

La période d'horloge est alors contrainte par :

$$T_{clk} \geq \text{temps}(\text{MUL}) + (\text{ORDER} + 1) \times \text{temps}(\text{ADD})$$

Ainsi, la fréquence maximale d'horloge est relativement faible. Cette architecture privilégie le débit (une sortie par cycle), mais au détriment de la vitesse maximale de fonctionnement.

Implémentation

L'architecture a été décrite en VHDL avec deux parties principales :

- un processus synchrone qui met à jour le registre à décalage `x_reg`,
- un processus combinatoire qui calcule la somme FIR à partir de `x_reg` et des coefficients `coeffs_i`.

Dans le processus combinatoire, une variable d'accumulation `acc_v` est utilisée pour sommer les produits :

$$x_reg(k) \cdot coeffs_i(k)$$

La largeur de l'accumulateur est augmentée avec la constante `ACC_W` afin de limiter les risques de dépassement pendant la somme.

Enfin, le résultat est décalé selon `COMMAPOS`, puis redimensionné sur `DATASIZE` bits pour produire `dout_o`.

Test(s) & Validation

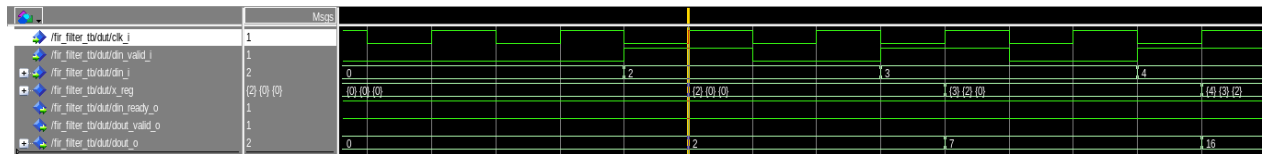
Log

```
do sim.do cmb
...
# Expected = 2 / Received = 2
# Expected = 7 / Received = 7
# Expected = 16 / Received = 16
# [PASSED] FIR combinatoire test successful.
```

Séquence

L'objectif de cette simulation est de vérifier le fonctionnement du filtre combinatoire sur un filtre d'ordre 2, avec les coefficients :

$$\text{coeffs_i}(0) = 1, \text{coeffs_i}(1) = 2, \text{coeffs_i}(2) = 3$$



La première donnée injectée est 2. Le registre à décalage contient alors cette valeur et la sortie obtenue est :

$$y = 2 \cdot 1 = 2$$

Ensuite, les valeurs 3 puis 4 sont injectées successivement. Comme le calcul est combinatoire, la sortie est recalculée directement à partir du contenu courant de x_reg .

Pour l'entrée 3, on obtient :

$$y = 3 \cdot 1 + 2 \cdot 2 = 7$$

Pour l'entrée 4, on obtient :

$$y = 4 \cdot 1 + 3 \cdot 2 + 2 \cdot 3 = 16$$

Les résultats observés correspondent aux valeurs attendues. Le chronogramme montre que $dout_o$ est mis à jour dès que le registre x_reg contient les nouvelles données.

Latence & Débit Le chronogramme montre que la sortie est mise à jour directement après le chargement de la donnée dans le registre à décalage.

Le calcul étant combinatoire, aucune phase de calcul sur plusieurs cycles n'est nécessaire.

$$\text{Latence} \approx 1 \text{ cycle}$$

Comme din_ready_o et $dout_valid_o$ restent constamment à 1, le filtre peut accepter une donnée à chaque cycle et produire une sortie à chaque cycle :

$$\text{Débit} = 1 \text{ sortie / cycle}$$

Utilisation des ressources

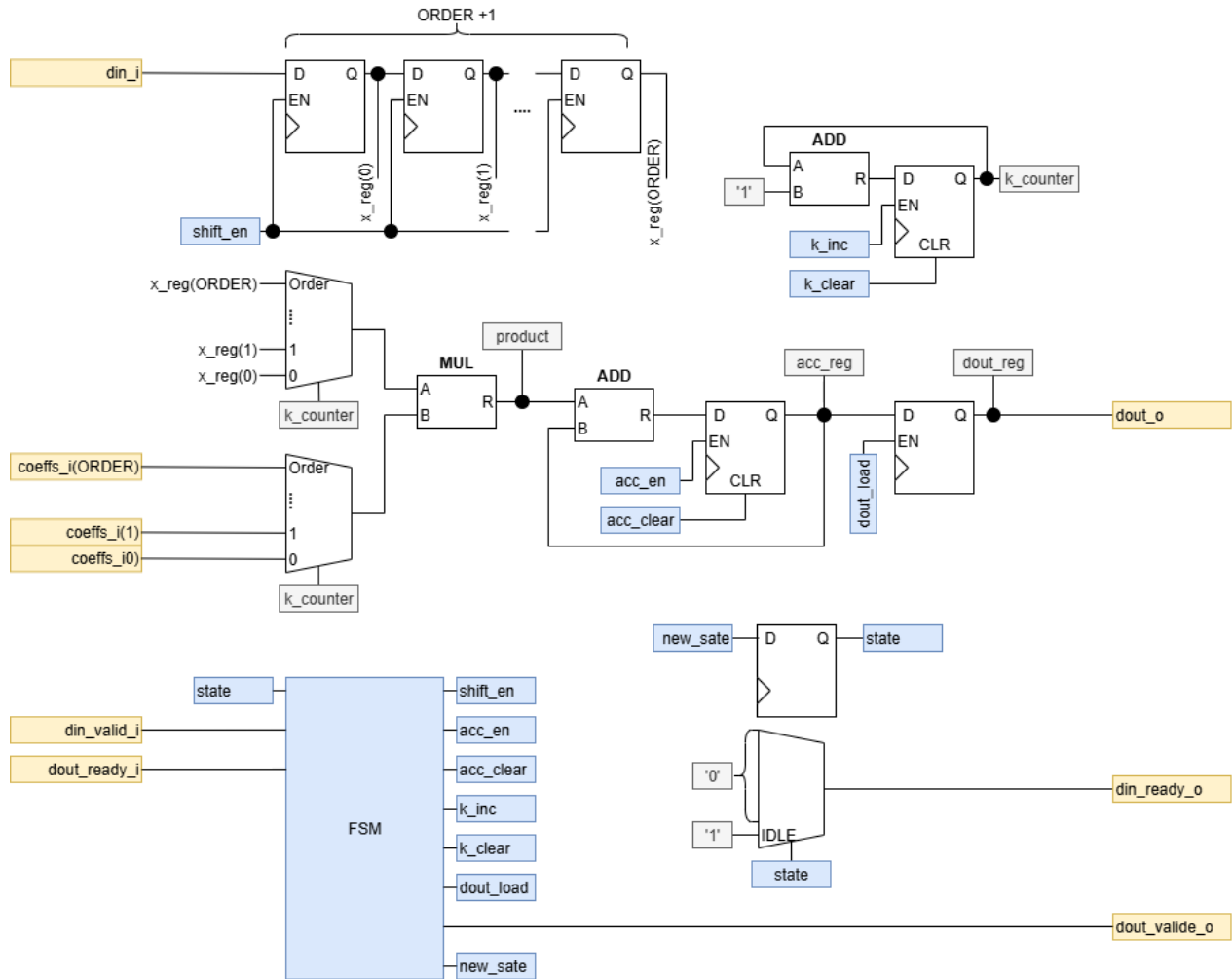
Les résultats de synthèse ont été obtenus avec les paramètres par défaut du filtre : $ORDER = 8$, $DATASIZE = 16$, $COEFFSIZE = 16$.

Flow Status	Successful - Mon May 4 18:43:20 2026
Quartus Prime Version	18.1.0 Build 625 09/12/2018 SJ Standard Edition
Revision Name	DE1_SoC_top
Top-level Entity Name	fir_filter_synth
Family	Cyclone V
Device	5CSEMA5F31C6
Timing Models	Final
Logic utilization (in ALMs)	106 / 32,070 (< 1 %)
Total registers	33
Total pins	38 / 457 (8 %)
Total virtual pins	0
Total block memory bits	0 / 4,065,280 (0 %)
Total DSP Blocks	6 / 87 (7 %)
Total HSSI RX PCSs	0
Total HSSI PMA RX Deserializers	0
Total HSSI TX PCSs	0
Total HSSI PMA TX Serializers	0
Total PLLs	0 / 6 (0 %)
Total DLLs	0 / 4 (0 %)

Filtre FIR : Séquentiel

Le filtre FIR est implémenté sous forme séquentielle, avec pour objectif de minimiser les ressources matérielles, en particulier en utilisant un seul multiplicateur.

Schéma



Chemin de données (datapath)

Les données d'entrée din_i sont stockées dans un registre à décalage (x_reg) de taille $ORDER + 1$, permettant de conserver les dernières valeurs reçues.

À chaque cycle, un échantillon $x_reg(k)$ et son coefficient $coeffs(k)$ sont sélectionnés via des multiplexeurs pilotés par un compteur $k_counter$. Leur produit est ensuite accumulé dans acc_reg . Après $ORDER + 1$ cycles, le résultat est chargé dans $dout_reg$ puis envoyé sur $dout_o$.

Unité de contrôle

Une FSM pilote le système avec quatre états :

- IDLE : attente d'une donnée

- MAC : accumulation
- LOAD : chargement du résultat
- OUTPUT : émission de la sortie

Elle génère les signaux de contrôle du datapath.

Interface

Le filtre utilise un protocole valid/ready :

- `din_ready_o` = 1 en IDLE → nouvelle donnée acceptée
- `dout_valid_o` = 1 en OUTPUT → sortie disponible

Latence & débit

Latence = ORDER + 2 cycles

Débit $\approx$ 1 sortie tous les (ORDER + 2) cycles

Fréquence d'horloge

Dans l'architecture séquentielle, le calcul est réparti sur plusieurs cycles. À chaque cycle, une seule opération de type multiplication-accumulation (MAC) est réalisée.

Le chemin critique est donc fortement réduit et correspond uniquement à :

$$T_{clk} \geq \text{temps}(\text{MUL}) + \text{temps}(\text{ADD})$$

Cela permet d'augmenter significativement la fréquence maximale d'horloge. En revanche, plusieurs cycles sont nécessaires pour produire une sortie, ce qui réduit le débit global.

Implémentation

L'architecture a été décrite en VHDL en séparant le datapath et l'unité de contrôle.

Le datapath comprend :

- un registre à décalage (`x_reg`) pour stocker les entrées,
- un multiplicateur et un additionneur pour le calcul MAC,
- un accumulateur (`acc_reg`) et un registre de sortie (`dout_reg`),
- un compteur (`k_counter`) pour parcourir les coefficients.

L'implémentation est structurée en deux processus principaux :

- un processus synchrone, déclenché sur le front montant de l'horloge, qui regroupe tous les registres du datapath (registre à décalage, accumulateur, compteur, sortie) ainsi que le registre d'état,
- un processus combinatoire, décrivant la FSM, qui détermine le prochain état et génère les signaux de contrôle (`shift_en`, `acc_en`, `k_inc`, etc.) en fonction de l'état courant et des entrées.

L'interface suit un protocole valid/ready, assurant une gestion correcte des échanges de données avec l'environnement.

Test(s) & Validation

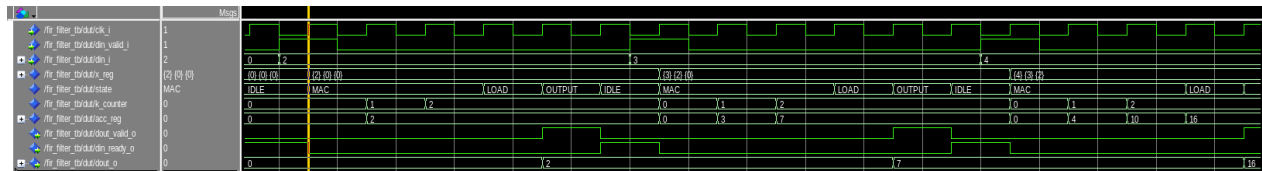
Log

```
do sim.do seq
...
# Expected = 2 / Received = 2
# Expected = 7 / Received = 7
# Expected = 16 / Received = 16
# [PASSED] FIR sequentiel test successful.
```

Séquence

L'objectif de cette simulation est de vérifier le fonctionnement temporel du filtre séquentiel sur un filtre d'ordre 2, avec les coefficients :

$$\text{coeffs_i}(0) = 1, \text{coeffs_i}(1) = 2, \text{coeffs_i}(2) = 3$$



La première donnée injectée est 2. Après le passage dans les états MAC, LOAD puis OUTPUT, la sortie obtenue est :

$$y = 2 \cdot 1 = 2$$

Ensuite, les valeurs 3 puis 4 sont injectées lorsque $\text{din_ready_o} = 1$. Le chronogramme montre que le compteur $k_counter$ parcourt successivement les coefficients et que les produits sont accumulés dans acc_reg .

Pour l'entrée 3, le registre contient les valeurs nécessaires au calcul :

$$y = 3 \cdot 1 + 2 \cdot 2 = 7$$

Pour l'entrée 4, les trois échantillons sont présents dans le registre à décalage :

$$y = 4 \cdot 1 + 3 \cdot 2 + 2 \cdot 3 = 16$$

Les résultats observés correspondent aux valeurs attendues. Le chronogramme montre également le fonctionnement correct de la FSM, avec les transitions entre les états IDLE, MAC, LOAD et OUTPUT.

Latence & Débit Le chronogramme montre qu'une sortie nécessite plusieurs cycles avant d'être disponible.

Pour un filtre d'ordre 2, le calcul effectue successivement les opérations MAC avant le chargement du résultat :

$$\text{Latence} \approx \text{ORDER} + 2 \text{ cycles}$$

Une nouvelle donnée ne peut être acceptée qu'après la fin du traitement précédent.

Le débit obtenu est donc :

$$\text{Débit} \approx 1 \text{ sortie tous les } (\text{ORDER} + 2) \text{ cycles}$$

Utilisation des ressources

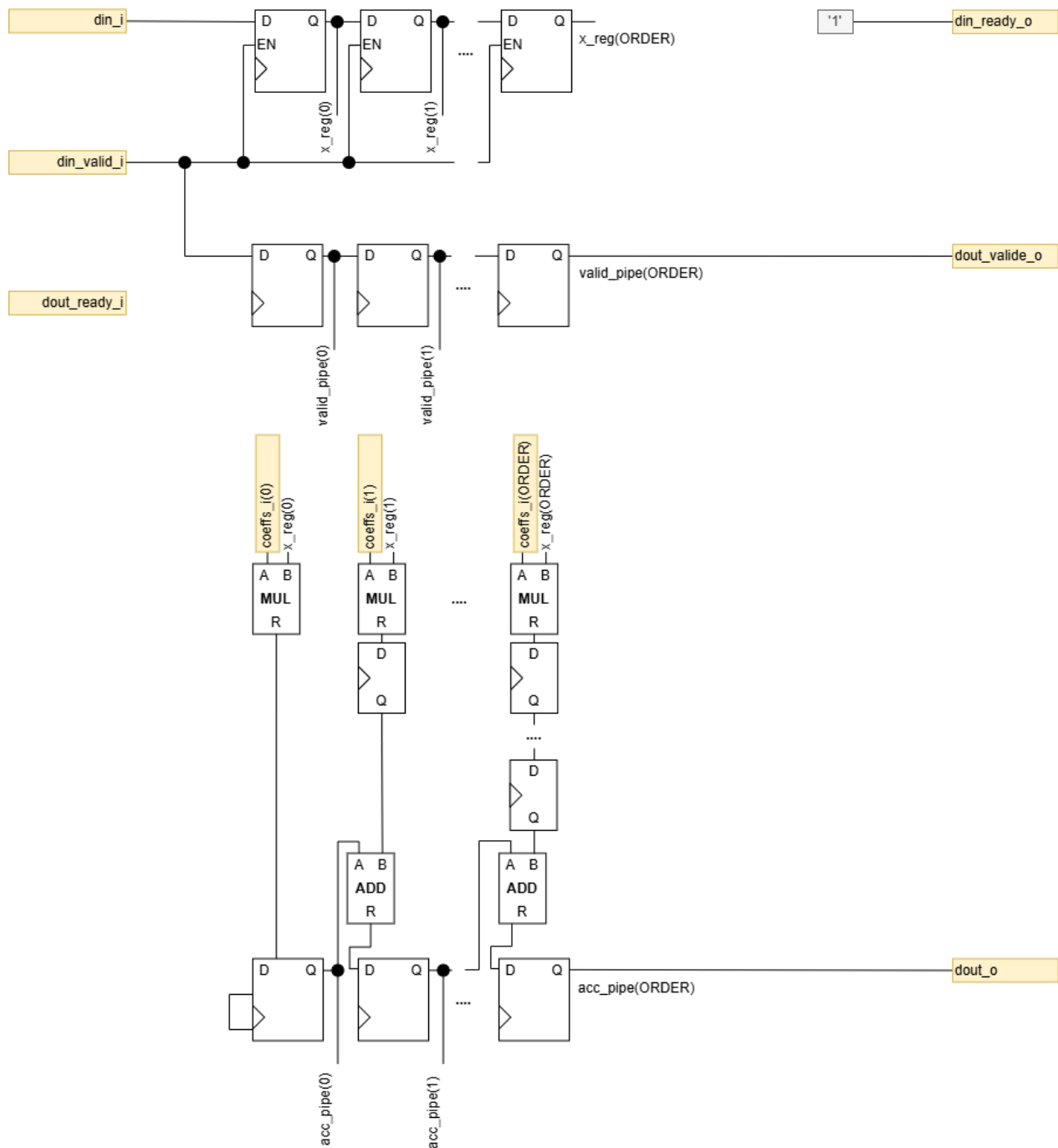
Les résultats de synthèse ont été obtenus avec les paramètres par défaut du filtre : ORDER = 8, DATASIZE = 16, COEFFSIZE = 16.

Flow Status	Successful - Mon May 4 18:40:18 2026
Quartus Prime Version	18.1.0 Build 625 09/12/2018 5J Standard Edition
Revision Name	DE1_SoC_top
Top-level Entity Name	fir_filter_synth
Family	Cyclone V
Device	5CSEMA5F31C6
Timing Models	Final
Logic utilization (in ALMs)	94 / 32,070 (< 1 %)
Total registers	224
Total pins	38 / 457 (8 %)
Total virtual pins	0
Total block memory bits	0 / 4,065,280 (0 %)
Total DSP Blocks	1 / 87 (1 %)
Total HSSI RX PCSs	0
Total HSSI PMA RX Deserializers	0
Total HSSI TX PCSs	0
Total HSSI PMA TX Serializers	0
Total PLLs	0 / 6 (0 %)
Total DLLs	0 / 4 (0 %)

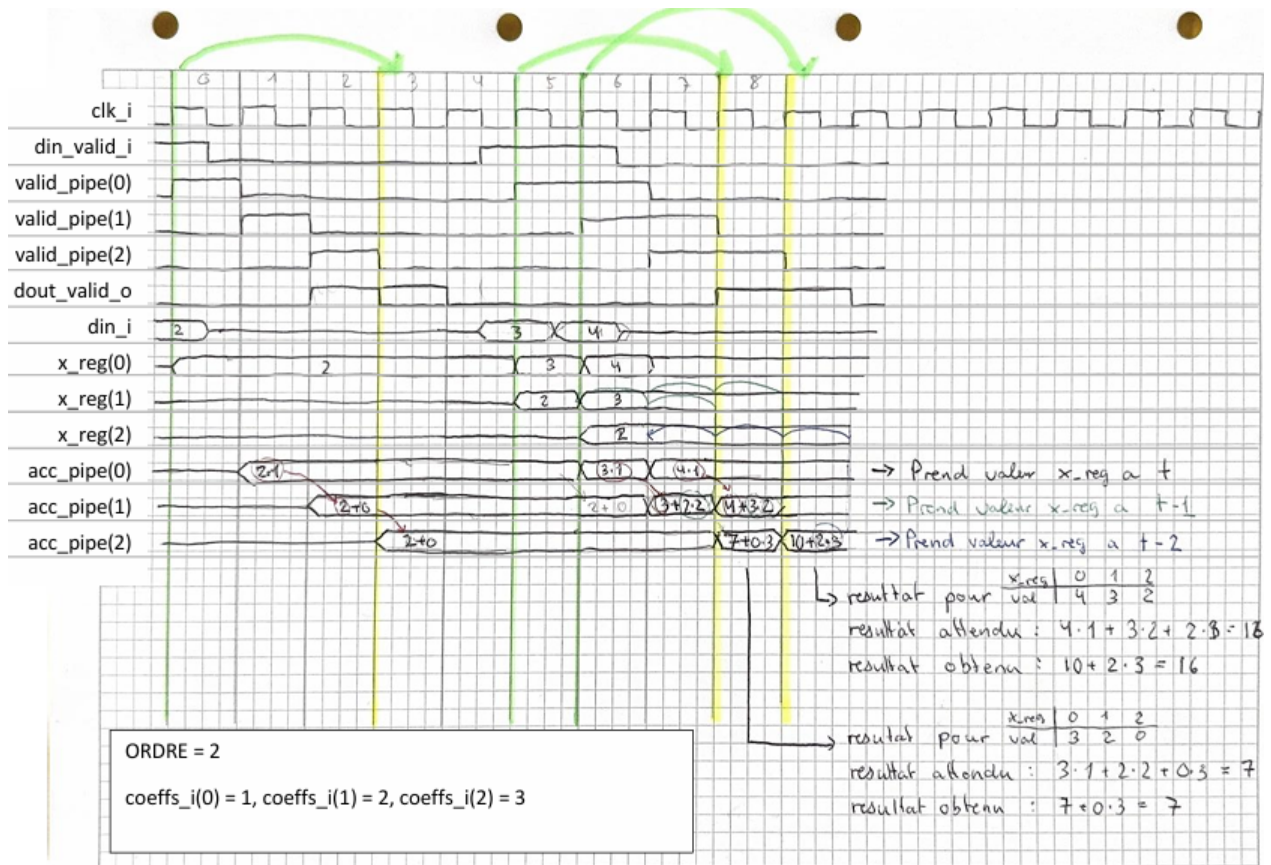
Filtre FIR : Pipeliné

Le filtre FIR est implémenté sous forme pipelinée, avec pour objectif d'augmenter la fréquence maximale d'horloge tout en conservant un débit élevé. Le calcul est réparti sur plusieurs étages séparés par des registres pipeline.

Schéma



Chronogramme



Chemin de données (datapath)

Les données d'entrée din_i sont stockées dans un registre à décalage x_reg de taille $ORDER+1$, permettant de conserver les derniers échantillons reçus.

Chaque échantillon retardé $x_reg(k)$ est multiplié par son coefficient associé $coeffs(k)$ à l'aide d'un multiplicateur dédié. Contrairement à l'architecture séquentielle, toutes les multiplications sont réalisées en parallèle.

Le calcul de la sortie FIR n'est cependant pas effectué en une seule étape. Il est réparti sur plusieurs étages de pipeline à l'aide d'un registre de sommes partielles acc_pipe .

Après chaque multiplication, des registres sont insérés afin de synchroniser les données dans le pipeline.

Cela permet à chaque échantillon d'avancer correctement d'un étage à chaque cycle, en conservant ses propres produits alignés dans le temps.

Par exemple, si l'on injecte successivement 3 puis 4 :

- le premier résultat correspond à $3 \cdot coeffs_i(0)$,
- le suivant à $4 \cdot coeffs_i(0) + 3 \cdot coeffs_i(1)$.

Les registres évitent ainsi de mélanger des produits provenant de différentes entrées, garantissant que chaque sortie correspond bien à une seule donnée d'entrée.

Interface

Le filtre utilise un protocole valid/ready simplifié :

- `din_ready_o` = 1 en permanence → le filtre accepte une donnée à chaque cycle,
- `dout_valid_o` est généré par le pipeline de validité,

Latence & débit

La latence correspond au nombre d'étages pipeline :

$$\text{Latence} = \text{ORDER} + 1 \text{ cycles}$$

En effet, une donnée doit traverser l'ensemble des étages de multiplication et d'accumulation avant de produire un résultat complet.

Cependant, une fois le pipeline rempli, le débit est maximal :

$$\text{Débit} = 1 \text{ sortie / cycle}$$

Cela signifie qu'une nouvelle donnée peut être injectée à chaque cycle, et qu'un résultat est produit à chaque cycle.

Fréquence d'horloge

Dans l'architecture pipelinée, le calcul FIR est réparti sur plusieurs étages séparés par des registres.

Le chemin critique est fortement réduit et correspond à une seule étape de calcul :

$$T_{clk} \geq \text{temps}(\text{MUL}) + \text{temps}(\text{ADD})$$

Chaque opération étant isolée par un registre, la fréquence maximale d'horloge est significativement plus élevée que dans la version combinatoire.

Cette architecture permet donc d'obtenir à la fois :

- un débit maximal (1 résultat par cycle),
 - une fréquence élevée.
-

Implémentation

L'architecture pipelinée a été décrite en VHDL autour d'un unique processus synchrone.

Le datapath comprend :

- un registre à décalage `x_reg` pour mémoriser les derniers échantillons,
- un pipeline d'accumulation `acc_pipe` pour stocker les sommes partielles,
- un pipeline de produits `prod_pipe` pour synchroniser les produits avec les bons étages,
- un pipeline de validité `valid_pipe`,
- un registre `dout_valid_reg` pour aligner `dout_valid_o` avec `dout_o`.

À chaque front montant de l'horloge, si `din_valid_i` = 1, la nouvelle donnée est chargée dans `x_reg(0)` et les anciennes valeurs sont décalées.

Les produits `x_reg(k) · coeffs_i(k)` sont ensuite calculés en parallèle. Les produits correspondant aux coefficients retardés sont stockés dans `prod_pipe`, afin d'arriver au bon moment dans les étages d'addition.

Le registre `acc_pipe(0)` contient le premier produit :

$x_reg(0) \cdot coeffs_i(0)$

Puis chaque étage suivant ajoute un produit synchronisé :

$acc_pipe(i) = acc_pipe(i-1) + prod_pipe(i, i-1)$

Le résultat final est pris depuis $acc_pipe(ORDER)$, puis décalé selon $COMMAPOS$ avant d'être envoyé sur $dout_o$.

Le signal $valid_pipe$ suit la progression des données dans le pipeline. Le signal $dout_valid_reg$ permet d'aligner $dout_valid_o$ avec la sortie $dout_o$.

Test(s) & Validation

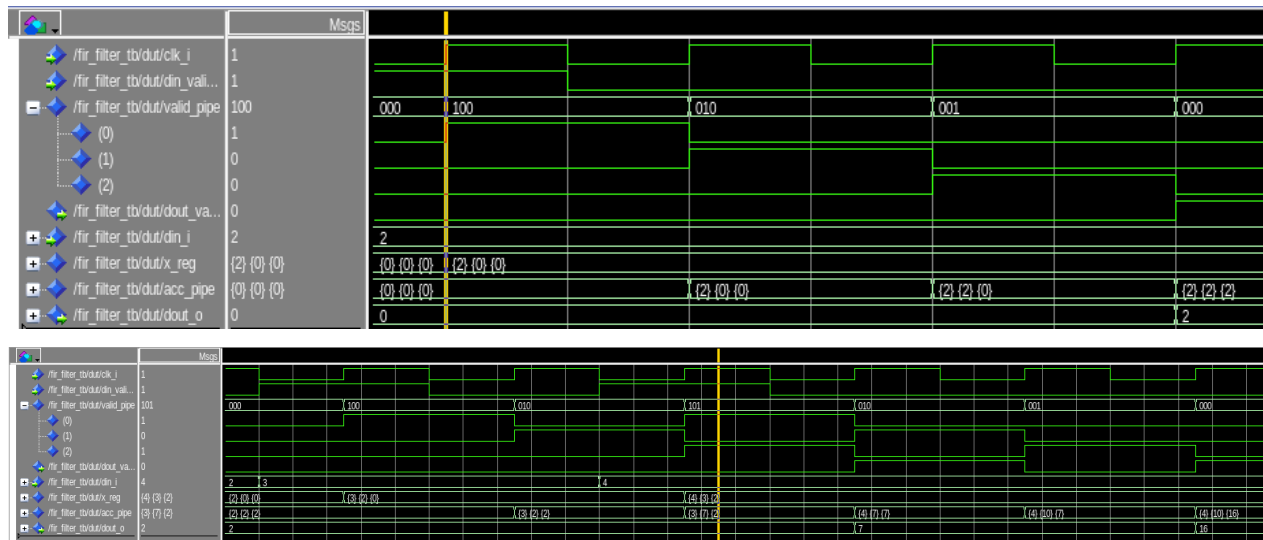
Log

```
do sim.do pip
...
# Expected = 2 / Received = 2
# Expected = 7 / Received = 7
# Expected = 16 / Received = 16
# [PASSED] FIR pipeline test successful.
```

Séquence

L'objectif de cette simulation est de vérifier le comportement temporel du pipeline sur un filtre d'ordre 2, avec les coefficients :

$coeffs_i(0) = 1, coeffs_i(1) = 2, coeffs_i(2) = 3$



La première donnée injectée est 2. Après propagation dans le pipeline, la sortie obtenue est :

$$y = 2 \cdot 1 = 2$$

Ensuite, les valeurs 3 et 4 sont injectées successivement. Le chronogramme montre que les données se décalent correctement dans x_reg et que les sommes partielles progressent dans acc_pipe .

Pour l'entrée 3, le registre contient les valeurs précédentes nécessaires au calcul :

$$y = 3 \cdot 1 + 2 \cdot 2 = 7$$

Pour l'entrée 4, les trois échantillons sont présents dans le filtre :

$$y = 4 \cdot 1 + 3 \cdot 2 + 2 \cdot 3 = 16$$

Les résultats observés dans la simulation correspondent aux valeurs attendues. Le signal `valid_pipe` permet de suivre la progression des données dans le pipeline, et `dout_valid_o` indique le moment où `dout_o` contient un résultat valide.

Le chronogramme confirme donc que les registres pipeline synchronisent correctement les produits et les sommes partielles, sans mélanger les contributions de différentes entrées.

Latence & Débit Le chronogramme montre qu'une donnée doit traverser les différents étages du pipeline avant de produire une sortie valide.

Pour un filtre d'ordre 2, la sortie apparaît après :

$$\text{Latence} = \text{ORDER} + 1 = 3 \text{ cycles}$$

On observe également que plusieurs données sont traitées simultanément dans le pipeline. Pendant qu'une donnée avance dans `acc_pipe`, une nouvelle entrée peut déjà être injectée.

Une fois le pipeline rempli, une sortie valide est produite à chaque cycle :

$$\text{Débit} = 1 \text{ sortie / cycle}$$

Utilisation des ressources

Les résultats de synthèse ont été obtenus avec les paramètres par défaut du filtre : `ORDER = 8`, `DATASIZE = 16`, `COEFFSIZE = 16`.

Flow Status	Successful - Thu May 7 14:44:38 2026
Quartus Prime Version	18.1.0 Build 625 09/12/2018 SJ Standard Edition
Revision Name	DE1_SoC_top
Top-level Entity Name	fir_filter_synth
Family	Cyclone V
Device	5CSEMA5F31C6
Timing Models	Final
Logic utilization (in ALMs)	360 / 32,070 (1 %)
Total registers	1188
Total pins	38 / 457 (8 %)
Total virtual pins	0
Total block memory bits	0 / 4,065,280 (0 %)
Total DSP Blocks	5 / 87 (6 %)
Total HSSI RX PCSs	0
Total HSSI PMA RX Deserializers	0
Total HSSI TX PCSs	0
Total HSSI PMA TX Serializers	0
Total PLLs	0 / 6 (0 %)
Total DLLs	0 / 4 (0 %)

Tableau comparatif

Critère	Combinatoire	Séquentiel	Pipeline
Multiplieurs	ORDER + 1	1 seul	ORDER + 1
Additionneurs	ORDER	1 seul	ORDER
Registres	33	224	1188
ALMs	106	94	360
DSP	6 / 87	1 / 87	5 / 87
Pins	38 / 457	38 / 457	38 / 457
FSM	Non	Oui	Non
Latence	≈ 1 cycle	ORDER + 2 cycles	ORDER + 1 cycles
Débit	1 sortie / cycle	1 sortie / (ORDER + 2) cycles	1 sortie / cycle
Fréquence max	Faible	Élevée	Très élevée
Adapté flux continu	Oui	Non	Oui

Analyse et conclusion

Les trois architectures FIR implémentées produisent les mêmes résultats fonctionnels, mais avec des compromis matériels différents.

La version combinatoire privilégie le débit : toutes les opérations sont réalisées en parallèle, ce qui permet d'obtenir une sortie à chaque cycle. En revanche, le chemin critique est long et la fréquence maximale reste limitée. Cette architecture utilise plusieurs DSP mais peu de registres.

La version séquentielle réduit fortement les ressources de calcul grâce à la réutilisation d'un seul multiplieur. Elle utilise donc peu de DSP, mais nécessite plusieurs cycles pour calculer une sortie. Le débit est plus faible, malgré une meilleure fréquence d'horloge.

La version pipelinée combine un débit élevé et une fréquence d'horloge importante grâce à l'insertion de registres pipeline entre les étages de calcul. Une fois rempli, le pipeline produit une sortie à chaque cycle. Cette solution est cependant la plus coûteuse en registres.

Les résultats de synthèse confirment ces compromis :

- combinatoire : plus de DSP, peu de registres,
- séquentiel : très peu de DSP, débit faible,
- pipeline : débit élevé et fréquence élevée, mais beaucoup de registres.

Le choix de l'architecture dépend donc principalement des contraintes du système : ressources disponibles, fréquence visée et débit nécessaire.